

Relatório de Bugs

ServeRest — front.serverest.dev & serverest.dev (API)

Levantamento de inconsistências e defeitos encontrados durante o desenvolvimento da suíte de automação Cypress (UI e API), com evidências reproduzidas manualmente e/ou via testes automatizados.

Componente	Bugs encontrados	Severidade Alta	Severidade Média/Baixa
UI (front.serverest.dev)	3	2	1
API (serverest.dev)	3	1	2

Bugs de UI (front.serverest.dev)

#1 — Formulário de login perde sincronia após uma tentativa falha

ALTA

Descrição: Após um login retornar 401 (credenciais inválidas), reenviar o mesmo formulário já renderizado com a senha corrigida falha com a mensagem "Email é obrigatório" — mesmo com o campo de e-mail visivelmente preenchido na tela. O valor exibido no DOM não está mais sincronizado com o estado interno do formulário depois do erro.

Evidência / como reproduzir:

1. Acessar `/login` com um e-mail válido e senha errada, clicar em Entrar (401).
2. Sem recarregar a página, apagar a senha e digitar a correta no mesmo formulário.
3. Clicar em Entrar novamente.

Resultado: erro "Email é obrigatório", apesar do campo mostrar o e-mail preenchido. Reproduzido manualmente (DevTools) e via teste Cypress automatizado.

Recomendação: Corrigir o gerenciamento de estado do formulário para não depender de re-sincronizar com o DOM após um envio malsucedido. Como contorno, o usuário (e a suíte de testes) precisa recarregar `/login` antes de tentar de novo.

#2 — Rota `/carrinho` inacabada em produção (placeholder "Em construção")

ALTA

Descrição: A página `/carrinho`, acessível pelo menu principal e destino natural do fluxo de compra (Lista de Compras → "Adicionar no carrinho"), não renderiza nenhum conteúdo funcional — exibe apenas o texto "Em construção aguarde". Um usuário que finaliza a montagem do carrinho não consegue ver os itens, valores ou concluir a compra pela interface.

Evidência / como reproduzir:

1. Logar, adicionar um produto à Lista de Compras.
 2. Clicar em "Adicionar no carrinho".
 3. A navegação leva a `/carrinho`, cujo conteúdo é apenas "Em construção aguarde".
- Confirmado via inspeção do texto renderizado da página em múltiplas execuções.

Recomendação: Finalizar a implementação da página de carrinho, ou remover/ocultar o link e o botão "Adicionar no carrinho" enquanto a funcionalidade não estiver disponível, para não expor um fluxo quebrado ao usuário final.

#3 — Área administrativa (`/admin/*`) acessível a qualquer usuário autenticado

MÉDIA

Descrição: As rotas `/admin/*` (incluindo cadastro/listagem de produtos e usuários) não verificam o papel (administrador) no front-end — qualquer usuário logado consegue navegar até a tela administrativa completa e ver toda a navegação exclusiva de admin. A API corretamente bloqueia a ação privilegiada (403 "Rota exclusiva para administradores"), mas a UI não esconde nem impede o acesso à tela em si, o que expõe funcionalidade e informação de layout que deveriam ser restritas.

Evidência / como reproduzir:

1. Criar/logar com um usuário comum (`administrador: false`).
2. Acessar diretamente a URL `/admin/home`.
3. A tela completa do admin é renderizada normalmente (`nav`, `cards`, etc.).
4. Ao tentar de fato cadastrar um produto, a API retorna 403 corretamente — mas a UI já havia exposto a tela inteira antes disso.

Confirmado manualmente e coberto por teste automatizado.

Recomendação: Adicionar um guard de rota no front-end que redirecione usuários não-administradores para `/home` (ou `/login`) ao tentar acessar qualquer rota `/admin/*`, em vez de depender somente da

validação da API.

Bugs de API (serverest.dev)

#4 — Senha do usuário retornada em texto puro (GET /usuarios e /usuarios/{id})

ALTA

Descrição: As rotas GET /usuarios e GET /usuarios/{id} incluem o campo "password" em texto puro no corpo da resposta. Exposição de credenciais em respostas de API é uma prática de segurança inadequada, mesmo em ambiente de treinamento — reforça o risco de nunca reutilizar senhas fixas/previsíveis em massa de teste.

Evidência / como reproduzir:

```
GET https://serverest.dev/usuarios/{id}
Resposta inclui: {"nome":..., "email":..., "password":"Teste@123", "administrador":..., "_id":...}
```

Confirmado via curl e via teste de API.

Recomendação: Nunca retornar o campo de senha (nem hash, nem texto puro) em nenhuma resposta de leitura de usuário. Se for mantido por ser um ambiente didático, ao menos documentar esse comportamento explicitamente no Swagger.

#5 — Validação de formato de ID mascara o erro "não encontrado" (limite de caracteres)

MÉDIA

Descrição: GET /usuarios/{id}, GET /produtos/{id} e GET /carrinhos/{id} exigem que o ID tenha exatamente 16 caracteres alfanuméricos. Um ID de tamanho diferente retorna um erro de formato ({"id": "id deve ter exatamente 16 caracteres alfanuméricos"}) em vez da mensagem esperada de "não encontrado" — mesmo que a intenção de quem consome a API seja simplesmente buscar um recurso inexistente. Essa regra de validação não está documentada no Swagger, então é fácil escrever um teste (ou uma integração) que use um ID "qualquer" (ex.: "idQueNaoExiste123", 17 caracteres) e receba um erro enganoso.

Evidência / como reproduzir:

```
GET https://serverest.dev/usuarios/idQueNaoExiste123 (17 caracteres)
→ 400 {"id": "id deve ter exatamente 16 caracteres alfanuméricos"}
```

```
GET https://serverest.dev/usuarios/a1b2c3d4e5f6g7h8 (16 caracteres, mas inexistente)
→ 400 {"message": "Usuário não encontrado"}
```

Mesmo comportamento confirmado em /produtos/{id} e /carrinhos/{id}.

Recomendação: Documentar essa regra de formato no Swagger (mínimo/máximo de caracteres do _id), ou unificar a resposta para sempre retornar "não encontrado" quando o ID não corresponder a um registro, independentemente do formato.

#6 — Nome do campo de erro diverge entre o Swagger e a resposta real da API

BAIXA

Descrição: O Swagger documenta o erro de "produto faz parte de carrinho" (DELETE /produtos/{id}) com o campo "idCarrinho" (singular, string). Na prática, a API retorna "idCarrinhos" (plural, array). Quem implementa uma integração baseada estritamente na especificação do Swagger vai ler o campo errado.

Evidência / como reproduzir:

```
Swagger (components.schemas.naoPermitidoExcluirProdutoDeCarrinho):
{"message": "...", "idCarrinho": "string"}
```

```
Resposta real (DELETE /produtos/{id} com produto em carrinho):
{"message": "...", "idCarrinhos": ["qbMqntef4iT0wWfg"]}
```

Confirmado via chamada direta à API comparada à especificação em

`serverest.dev/swagger.json`.

Recomendação: Atualizar a documentação do Swagger para refletir o nome e o tipo reais do campo ("idCarrinhos", array), evitando integrações quebradas por confiarem na especificação.
